

2026-05-06-p2-f21-codex-approval-modes-design

Phase 2 / Feature 21 — Codex Approval Modes (FIRST Phase 2 feature)

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 2 port (codex / cursor / aider patterns)

Note (programme position): F21 is the **first** Phase 2 feature. T01 (bootstrap) flips PROGRESS.md from “Phase 1 complete” to “Phase 2 in flight”; T09 (close-out) records the first Phase 2 evidence section.

1. Goal

Ship a real, end-to-end **4-mode codex-compatible approval system** for the HelixCode CLI agent so that mutating tool calls (file edits, shell commands, MCP side-effects) are gated against an explicit user-chosen risk posture instead of either always-prompting (current default) or always-allowing (the silent bluff that broke earlier revisions of this codebase). The four modes are taken verbatim from codex (`cli_agents/codex/`): `suggest`, `auto-edit`, `full-auto`, `dangerously-bypass` (Q1=A). Each mode has clear, byte-deterministic semantics; switching modes at runtime via the `/approval` slash command must take effect on the very next tool call, not the call after that.

Three concrete user surfaces ship together:

1. approval **package** (`helix_code/internal/approval/`) — `ApprovalMode` enum (4 values: `ModeSuggest`, `ModeAutoEdit`, `ModeFullAuto`, `ModeDangerous`) + `ApprovalLevel` enum (4 values: `LevelReadOnly`, `LevelEdit`, `LevelRun`, `LevelAll`) + `Decision` value (`Allow/Deny/Prompt` + reason) + `ApprovalManager` (resolves the active mode, calls `tool.RequiresApproval()`, consults F02’s permission engine FIRST, then enforces the mode gate, then for `full-auto` + `LevelRun` rewires shell calls through F14’s sandbox manager with network DENY forced on) + `Selector` resolving flag `> env > config > default suggest` (Q2=C; mirrors F12’s `SelectorInput`). Pure-function selector; no `env` reads inside the function body — `env` closure injected via `SelectorInput.Env`.
2. `Tool.RequiresApproval()` **interface extension** (Q3=B) — `internal/tools/registry.go`’s `Tool` interface gains a new method `RequiresApproval()` `approval.ApprovalLevel`. Existing tools without an implementation default to `LevelEdit` via a one-line embedding helper (`approval.DefaultLevelEdit`) injected during a one-pass migration in T05; read-only tools (LSP queries, file reads, repomap dumps, mapping queries, browser reads) override to `LevelReadOnly`; shell/exec tools override to `LevelRun`; subagent dispatch + permission-rule writes override to `LevelAll`. The registry’s `Execute` method (already a hook point for F02 + F14) acquires one new pre-execute branch: `manager.CheckApproval(ctx, tool, params)` returning `Decision{Allow|Deny|Prompt, Reason}`; `Deny`

propagates as `ErrApprovalRequired` with the operative-mode hint; `Prompt` calls into F19's `Prompter` (already resident on `*CLI`) to read y/n; `Allow` proceeds to the existing F02 + F14 path.

3. `/approval slash command + CLI flag + env var` (Q5=A) — `--approval=<mode>` CLI flag (highest precedence); `HELIXCODE_APPROVAL=<mode>` env var; `~/.config/helixcode/approval.yaml` config file (lowest user-supplied precedence; reuses F12's wizard-writer YAML loader pattern); default suggest. `/approval slash command` with three subcommands: `/approval status` (active mode + source + per-mode summary line), `/approval set <mode>` (mutates the active manager's mode at runtime via an `atomic.Value` swap; takes effect on the very next tool call), `/approval show <mode>` (prints the four-line semantic summary of `<mode>` per §3.4). NO cobra subcommand.

The four-mode F14-coupling matrix (Q4=A) is **structurally enforced**: the `ApprovalManager.CheckApproval` consults the mode and the tool's `RequiresApproval()` level via a single 4×4 lookup table (§4.2); transitions that should “force the sandbox on” do so by mutating the shell tool's invocation path before `tool.Execute` runs (the manager wraps the params with a sandbox-required marker that the registry's pre-execute branch reads and routes through `sandbox.Manager.Execute(ctx, command, SandboxPolicy{NetworkAllowed: false})`). A unit test asserts `ModeFullAuto + LevelRun ⇒ Decision.Allow` AND the params map carries `_helix_sandbox_required: true` AND `NetworkAllowed=false`. A second unit test asserts `ModeSuggest + LevelEdit ⇒ Decision.Deny` with reason “approval required: switch to auto-edit or higher” and that `tool.Execute` is NEVER called (assertion via a fake tool counting `Execute` invocations; counter must remain at 0).

Runtime mode change via `/approval set <mode>` (Q5=A) is **structurally enforced** to take effect on the very next tool call: the `ApprovalManager` holds the active mode in an `atomic.Pointer[ApprovalMode]`; `Set(mode ApprovalMode)` swaps the pointer; `CheckApproval` loads via `Load()` on entry; a unit test runs `manager.Set(ModeAutoEdit)` then `manager.CheckApproval(ctx, fakeEditTool, params)` in the same goroutine and asserts `Decision.Allow` (where the prior call with `ModeSuggest` produced `Decision.Deny`).

The single largest bluff vector for F21 is “**approval mode loaded from config but never enforced at the registry boundary**” — a mode is shown in `/approval status`, the `internal/approval/` package compiles + has 100% unit-test coverage, but `registry.Execute` never calls `manager.CheckApproval`, so every tool runs regardless of mode. §5.2 enumerates four such patterns and pins each with a positive-evidence test + Challenge phase. The Challenge harness MUST exit non-zero on byte-level mismatch (e.g., `ModeSuggest` + a real `filesystem.WriteFileTool` invoked through the real registry must return `ErrApprovalRequired` AND the target file MUST NOT exist on disk after the call returns; the harness fails if either side is wrong).

Out of scope for v1: auto-promotion (e.g., 3 successful auto-edit cycles → propose full-auto); per-action audit log (F02's confirmation audit is reused; F21 does NOT add a parallel audit stream); multi-user RBAC (single-user CLI); persistence of mode across sessions beyond the YAML file (no automatic write-back from `/approval set`); per-tool approval overrides via YAML (e.g., “always allow `read_file` even in suggest” — already handled by `RequiresApproval()` returning `LevelReadOnly`). See §8.

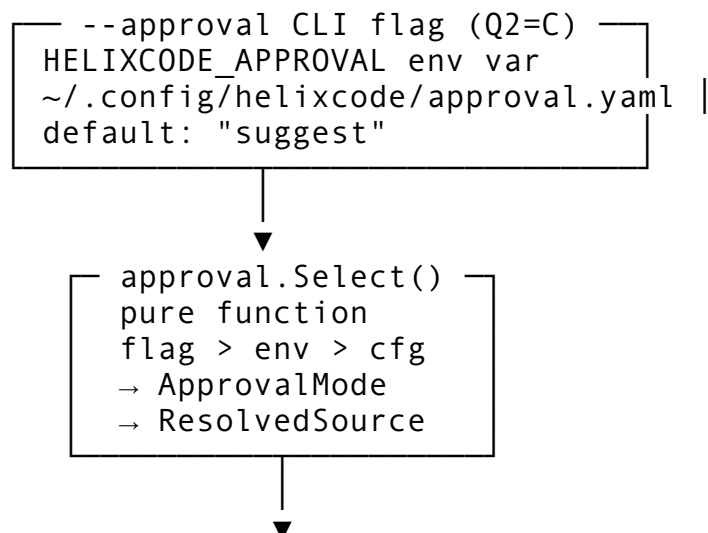
Anti-bluff hot zone (loud): an approval mode loaded but never enforced (the test asserts `manager.Mode() == ModeSuggest` but never wires the manager to the registry's pre-execute path, so a real `WriteFileTool.Execute` call still mutates disk); suggest mode

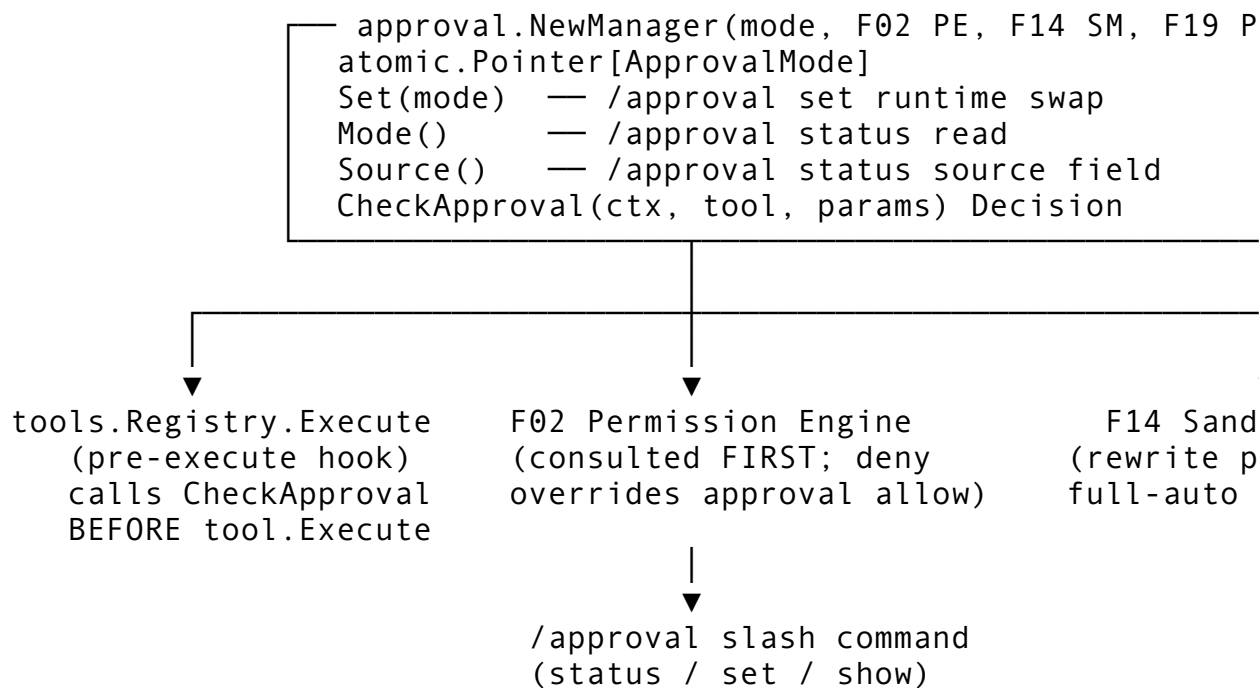
says “would do X” but actually executes silently (the gate produces a `Decision.Deny` value but the registry ignores it and proceeds); `full-auto` claims sandbox-on but `RequiresApproval()` doesn’t actually consult the sandbox manager (the `params` marker is set but `sandbox.Manager.Execute` is never invoked; the shell command runs through plain `os/exec` with full network); mode change at runtime via `/approval` slash isn’t reflected in subsequent tool calls (the slash mutates a struct field that `CheckApproval` doesn’t re-read). Each of these maps to a unit + integration + Challenge phase per §5.2.

2. Architecture

Four layers, all under `helix_code/internal/approval/`, plus thin wiring at the registry boundary, the F02 + F14 integration points, and one slash command:

- **ApprovalMode enum + ApprovalLevel enum + Decision value type** (`types.go`) — pure value types; the mode and level are `int` so the 4×4 lookup table is a `[4][4]Action` array (zero allocations).
- **Selector** (`selector.go`) — pure function `Select(SelectorInput)` (`ApprovalMode, ResolvedSource, error`) resolving `flag > env > config > default` suggest. Mirrors F12’s `Select` shape verbatim. `SelectorInput.Env` `func(string) string` so unit tests inject closures (no `os.Getenv` in the function body).
- **ApprovalManager** (`manager.go`) — runtime façade. Holds an `atomic.Pointer[ApprovalMode]` for the active mode, a `*confirmation.PolicyEngine` reference to F02 (for the “permission engine wins over approval” rule), a `*sandbox.Manager` reference to F14 (for the `full-auto + LevelRun` sandbox-required rewrite), and a `Prompter` (reuses F19’s `askuser.Prompter` interface) for `auto-edit + LevelRun` y/n prompts. Public API: `CheckApproval(ctx, tool Tool, params) Decision, Mode()` `ApprovalMode, Set(mode ApprovalMode), Source() ResolvedSource`.
- **Tool.RequiresApproval() interface extension** (extends `internal/tools/registry.go`) — every tool exposes `RequiresApproval()` `approval.ApprovalLevel`. A default-edit helper struct (`approval.DefaultLevelEdit`) is embedded by tools that don’t override; T05’s migration covers all existing tools (~30+) with explicit overrides on read-only tools (LSP query, file read, `repomap`, `mapping`, `browser` read), shell/exec tools (`LevelRun`), and high-risk tools (subagent dispatch, permission rule mutations: `LevelAll`).





Wire points (existing code; one addition per location):

- `internal/tools/registry.go::Execute` — adds a new pre-execute branch BEFORE the existing F02 hook + F14 dispatch: `if r.approvalMgr != nil { dec := r.approvalMgr.CheckApproval(ctx, tool, params); switch dec.Action { case ActionDeny: return nil, fmt.Errorf("%w: %s", ErrApprovalRequired, dec.Reason); case ActionPrompt: ... case ActionAllow: }`. The branch is the **first** pre-execute check (before F02), so F02's permission engine sees only the parameter set the user has already approved at the mode level. Note (§4.4): F02 STILL has final-deny authority — F02 returning `ActionDeny` is honoured even when F21 returned `ActionAllow`. F21 is “the user opted into this risk posture”; F02 is “this specific operation is forbidden”. They compose monotonically: any deny at any layer denies.
- `internal/tools/registry.go` **field**: new optional field `approvalMgr`
*`approval.ApprovalManager` (nil-safe — when nil, all calls bypass approval gate, preserving backward compatibility for tests + the `cmd/server` HTTP path that doesn't run the interactive CLI).

- `internal/tools/registry.go::Tool` **interface** — gains one method:

```

// RequiresApproval reports the approval level required to
// execute this
// tool. LevelReadOnly bypasses the gate entirely (tools that
// only read
// state — file reads, LSP queries, repomap dumps); LevelEdit
// gates the
// tool behind ModeAutoEdit or higher; LevelRun gates behind
// ModeFullAuto
// (with sandbox forced) or behind interactive y/n in
// ModeAutoEdit;
// LevelAll gates behind ModeDangerous (escape hatch only).
RequiresApproval() approval.ApprovalLevel
  
```

- `cmd/cli/main.go::run` — three additions:
 1. Construct `approval.NewSelector(...)` + resolve mode (with `--approval` CLI flag wired into the existing pflag set, `HELIXCODE_APPROVAL` env, YAML file path).
 2. Construct `approval.NewManager(mode, c.policyEngine, c.sandboxMgr, c.prompter)`.
 3. Wire `c.toolRegistry.SetApprovalManager(c.approvalMgr)` (one new method on the registry).
- `internal/commands/approval_command.go` — new file. Exposes `NewApprovalCommand(mgr *approval.ApprovalManager)` returning a `Command`. Three subcommands: `status` / `set <mode>` / `show <mode>`. `set` calls `mgr.Set(mode)`; `show` reads the static descriptor table in `internal/approval/types.go` (no manager needed for `show` — pure data).
- **Existing tools** (~30 across `internal/tools/`) — receive a single-line `RequiresApproval()` method per type. Tools that don't override embed `approval.DefaultLevelEdit` (a tiny struct with the method) for the safe default. T05 enumerates each tool's chosen level + rationale.

Why a new `internal/approval/` package and not “extend `internal/tools/permissions/`”: - F02's permission engine is **rule-based** (path globs, command patterns, deny-list). F21's mode is **risk-posture-based** (a single enum that gates whole categories). They serve different purposes; conflating them would either (a) bloat F02's rules with mode-as-a-rule (every rule needs a mode predicate), or (b) bloat F21's manager with rule semantics (defeating the simple 4×4 table). Keeping them separate lets F02 handle “this `rm -rf /etc` is denied by rule” and F21 handle “you're in `suggest` mode so no edits regardless of rules”. - F02 stays unchanged structurally (no schema migration of rules YAML), so F21 ships without breaking F02's existing test suite. - F02's policy engine is reused for the runtime path: F21's manager calls `policyEngine.Evaluate(req)` AFTER its own gate decided `Allow`, so F02 retains final-deny authority (§4.4).

Why per-tool category gate (Q3=B) and not centralised: - A tool knows what it does (a read tool knows it reads; a write tool knows it writes). Forcing a central registry to declare “tool X is dangerous” duplicates knowledge and risks drift when a new tool is added without updating the registry. - The default (`DefaultLevelEdit`) is safe — if a developer forgets to override, the tool is gated MORE strictly (`LevelEdit`), not less. The migration in T05 explicitly downgrades read-only tools to `LevelReadOnly` (which bypasses the gate); a forgotten override degrades to safe-by-default behaviour. - A unit test in T05 enumerates every tool in the registry's `buildToolList` and asserts its `RequiresApproval()` matches the expected level (table-driven; rationale documented in §3.6). This pins the migration and detects future tool additions that forgot to override.

Why slash + flag + env + config (Q5=A) and not cobra: - Approval mode is a *runtime posture* — the user wants to flip from `suggest` to `auto-edit` mid-session (“OK, I've reviewed the proposal, go ahead”). A cobra subcommand would force a process restart. - The CLI flag covers “I know what I'm doing for this single invocation”; the env var covers “all my CI runs are full-auto”; the YAML config covers “my default posture across all sessions”; the slash covers “switch mid-session”. Each surface targets a distinct ergonomic case. - A cobra subcommand `helixcode approval` would duplicate the slash; the slash already covers the user-facing case. F21.5 may add a debug-only cobra command for scripted batch mode-flips, but v1 keeps the surface area minimal.

3. Components

3.1 New files

- `helix_code/internal/approval/types.go` — `ApprovalMode`, `ApprovalLevel`, `Action`, `Decision`, `ResolvedSource`, sentinel errors (`ErrApprovalRequired`, `ErrUnknownMode`, `ErrSandboxRequired`), constants, mode descriptors (`ModeDescriptor` table for `/approval show`).
- `helix_code/internal/approval/types_test.go`.
- `helix_code/internal/approval/selector.go` — `SelectorInput`, `Select(input)` (`ApprovalMode`, `ResolvedSource`, error), `ParseMode(s string)` (`ApprovalMode`, error).
- `helix_code/internal/approval/selector_test.go` — table-driven over (flag, env, config) precedence + every mode-string parse case + unknown-mode error.
- `helix_code/internal/approval/manager.go` — `ApprovalManager`, `NewManager(opts ManagerOptions)`, `CheckApproval(ctx, tool, params)` `Decision`, `Set(mode)`, `Mode()`, `Source()`, `DefaultLevelEdit` helper struct.
- `helix_code/internal/approval/manager_test.go` — 4×4 mode×level matrix + F02 final-deny composition + F14 sandbox-required marker + atomic-swap runtime change + Prompter integration for auto-edit + `LevelRun`.
- `helix_code/internal/approval/yaml_loader.go` — `LoadConfigFile(path string)` (`ApprovalMode`, error); mirrors F12's `wizard_writer.go::LoadWizardConfig` style. File mode 0644 OK (no secrets — the chosen mode is not sensitive).
- `helix_code/internal/approval/yaml_loader_test.go` — real `tempdir` + `os.WriteFile` + parse 4 valid + 1 invalid YAML.
- `helix_code/internal/commands/approval_command.go` — `ApprovalCommand(Command impl)`; `/approval status`, `/approval set <mode>`, `/approval show <mode>`.
- `helix_code/internal/commands/approval_command_test.go`.
- `helix_code/tests/integration/approval_test.go` — `//go:build integration`; ALWAYS-runs; real registry + real F02 PE + real F14 SM + real Prompter (or fake-tty Prompter); per-mode behaviour assertions.
- `helix_code/tests/integration/cmd/p2f21_challenge/main.go` — runtime evidence harness.
- `challenges/p2-f21-codex-approval-modes/CHALLENGE.md` + `run.sh`.

3.2 Modified files

- `helix_code/internal/tools/registry.go` — three additions: (1) `Tool` interface gains `RequiresApproval()` `approval.ApprovalLevel`; (2) `ToolRegistry` struct gains `approvalMgr *approval.ApprovalManager` field + `SetApprovalManager(mgr)` setter; (3) `Execute` gains a new pre-execute branch (FIRST in the chain) calling `manager.CheckApproval` and short-circuiting on Deny or routing through Prompter on Prompt.
- `helix_code/internal/tools/**/*.go` — every existing tool gains a one-line `RequiresApproval()` method (~30 tools across filesystem, shell, browser, mapping, multiedit, lsp, sandbox, plan, askuser, smartedit, subagent, mcp, web, git, notebook, interactive). T05 enumerates them in §3.6 with the explicit level + rationale per tool. Tools without an explicit override embed `approval.DefaultLevelEdit` for the safe default.

- `helix_code/cmd/cli/main.go` — three additions: (1) construct `approval.Selector` + resolve mode at startup; (2) construct `approval.ApprovalManager` adjacent to the F02 + F14 + F19 wiring; (3) `c.toolRegistry.SetApprovalManager(c.approvalMgr) + register / approval slash`; (4) one new pflag `--approval=<mode>` wired into the existing flag set.
- `helix_code/internal/commands/registry.go` — no schema change; one new `Register(...)` call site for `/approval`.
- `helix_code/go.mod` — **zero new external deps**. `gopkg.in/yaml.v3` is already a direct dep (promoted in F20). `sync/atomic` is `stdlib`.

3.3 Types

```
// internal/approval/types.go

// ApprovalMode is the user's chosen risk posture. Four values
// verbatim from
// codex (cli_agents/codex/). Zero value (ModeSuggest) is the
// safe default.
type ApprovalMode int

const (
    ModeSuggest    ApprovalMode = iota // read-only; mutating
        tool calls REJECTED
    ModeAutoEdit           // edits OK; runs prompt
        y/n
    ModeFullAuto           // edits + runs OK; runs
        go through F14 sandbox
    ModeDangerous          // no checks (dangerously-
        bypass); user accepts full responsibility
    numModes
)

// String returns the codex-compatible mode string ("suggest",
// "auto-edit",
// "full-auto", "dangerously-bypass") for use in --approval flag
// values, env
// var values, YAML keys, /approval status output, and error
// messages.
func (m ApprovalMode) String() string

// ParseMode maps a flag/env/YAML string back to an ApprovalMode.
// Returns
// ErrUnknownMode for any unknown string. Accepts both kebab-case
// and
// underscore variants for ergonomics ("auto-edit" and
// "auto_edit" both
// resolve to ModeAutoEdit).
func ParseMode(s string) (ApprovalMode, error)
```

```

// ApprovalLevel is the approval requirement of a single tool.
// Tools declare
// their own level via Tool.RequiresApproval(). The 4x4 lookup
// table in
// §4.2 maps (Mode, Level) → Action.
type ApprovalLevel int

const (
    LevelReadOnly ApprovalLevel = iota // pure reads (file read,
        LSP query, repomap)
    LevelEdit // file mutations (write,
        multi-edit, smart-edit)
    LevelRun // shell/exec (sandbox-
        eligible)
    LevelAll // high-risk (subagent
        dispatch, permission writes)
    numLevels
)

// String returns "read-only" / "edit" / "run" / "all" for /
// approval show output.
func (l ApprovalLevel) String() string

// Action is the atomic decision for a single (Mode, Level) pair.
// Three
// values: Allow proceeds; Deny short-circuits with
// ErrApprovalRequired;
// Prompt asks the user via F19's Prompter for y/n.
type Action int

const (
    ActionAllow Action = iota // proceed to F02 → F14 →
        tool.Execute
    ActionDeny // short-circuit with
        ErrApprovalRequired
    ActionPrompt // ask user y/n via F19 Prompter;
        on "n" → Deny
)

// String returns "allow" / "deny" / "prompt" for /approval show
// + log output.
func (a Action) String() string

// Decision is the result of ApprovalManager.CheckApproval.
// Reason is a
// human-readable string suitable for the ErrApprovalRequired
// message and

```



```

// the /approval show output. SandboxRequired is true when (Mode,
// Level) ==
// (FullAuto, Run) and the registry must rewrite the tool call to
// go
// through F14's sandbox.Manager with NetworkAllowed=false.
type Decision struct {
    Action          Action
    Reason           string
    SandboxRequired bool // true ⇒ registry routes through F14
                      with NetworkAllowed=false
}

// ResolvedSource records WHERE the active mode came from. Used
// by
// /approval status output. One of "flag" / "env" / "config" /
// "default".
type ResolvedSource string

const (
    SourceFlag    ResolvedSource = "flag"
    SourceEnv     ResolvedSource = "env"
    SourceConfig  ResolvedSource = "config"
    SourceDefault ResolvedSource = "default"
)

// ModeDescriptor is the static descriptor for /approval show
// <mode>. One
// entry per mode; the table is exported as ModeDescriptors for
// the slash
// command's lookup. Each descriptor lists what the mode allows +
// denies
// per level + the F14 sandbox coupling.
type ModeDescriptor struct {
    Mode          ApprovalMode
    Summary       string           // one-line summary for /approval
                      status
    Description    string           // multi-line description for /
                      approval show
    Matrix
        [numLevels]Action // per-level action (the (Mode, *) row
                      of §4.2)
}

var ModeDescriptors = [numModes]ModeDescriptor{ /* see §3.4 */ }

// Sentinel errors. Tests compare via errors.Is.
var (

```

```

    ErrApprovalRequired =
        errors.New("approval: mode forbids this operation")
    ErrUnknownMode      = errors.New("approval: unknown mode")
    ErrSandboxRequired  = errors.New("approval: full-auto
        requires a working sandbox backend")
)

// internal/approval/selector.go

type SelectorInput struct {
    Flag      string          // --approval=<value>; ""
              means flag not set
    Env       func(string)
              string // env lookup closure (default os.Getenv)
    ConfigPath string        // ~/.config/helixcode/
              approval.yaml; "" means no config file
    Filesystem
        fs.FS          // injected for tests; default
        os.DirFS("/")
}

// Select resolves the active mode per flag > env > config >
// default precedence.
// Pure function: no env reads inside the function body (uses
// input.Env), no
// disk I/O outside loadConfigFile (which itself uses
// input.Filesystem).
func Select(input SelectorInput) (ApprovalMode, ResolvedSource,
    error)

// internal/approval/manager.go

type ManagerOptions struct {
    Mode          ApprovalMode
    Source         ResolvedSource
    PolicyEngine  *confirmation.PolicyEngine // F02
    SandboxMgr    *sandbox.SandboxManager    // F14
    Prompter      YesNoPrompter           // F19 (or wrapper
                          around its Prompter)
    Logger        *zap.Logger                // optional; nil →
                          no-op
}

type YesNoPrompter interface {
    PromptYesNo(ctx context.Context, question string, defaultYes
        bool) (bool, error)
}

type ApprovalManager struct {

```

```

    mode      atomic.Pointer[ApprovalMode]
    source     ResolvedSource
    pe         *confirmation.PolicyEngine
    sm         *sandbox.SandboxManager
    prompter   YesNoPrompter
    log        *zap.Logger
}

func NewManager(opts ManagerOptions) *ApprovalManager

// CheckApproval is the single gate. The registry calls this
// BEFORE F02 +
// F14 + tool.Execute. Returns Decision{Allow|Deny|Prompt,
// Reason,
// SandboxRequired}. Concurrency-safe (mode read via
// atomic.Pointer.Load).
func (m *ApprovalManager) CheckApproval(ctx
    context.Context, tool Tool, params
    map[string]interface{}) Decision

// Mode returns the active mode (atomic load).
func (m *ApprovalManager) Mode() ApprovalMode

// Set swaps the active mode atomically. Used by /approval set
// <mode>.
// Takes effect on the very next CheckApproval call.
func (m *ApprovalManager) Set(mode ApprovalMode)

// Source returns the source of the original mode resolution
// (flag / env /
// config / default). NOT updated by Set – Set always reports its
// source as
// "runtime"; the field on the manager is "as last resolved at
// startup".
func (m *ApprovalManager) Source() ResolvedSource

// Tool is the minimal subset of tools.Tool that ApprovalManager
// needs.
// Defined here to avoid an import cycle with internal/tools.
type Tool interface {
    Name() string
    RequiresApproval() ApprovalLevel
}

// DefaultLevelEdit is a tiny embeddable struct that provides a
// safe-default
// RequiresApproval method. Tools that don't have a clear level
// can embed

```

```
// this; the safe default is LevelEdit (the "ask before mutating"
    tier).
type DefaultLevelEdit struct{}

func (DefaultLevelEdit) RequiresApproval() ApprovalLevel {
    return LevelEdit }
```

3.4 Mode descriptors (concrete, load-bearing)

The four modes verbatim from codex (`cli_agents/codex/`), with their full per-level action matrix. Tests in T02 + T04 pin these byte-for-byte. The 4×4 matrix is the **only** source of truth for `CheckApproval`'s decision — no per-tool special-casing.

(Mode → / Level ↓)	suggest	auto-edit	full-auto	dangerously-bypass
read-only	Allow	Allow	Allow	Allow
edit	Deny	Allow	Allow	Allow
run	Deny	Prompt	Allow + sandbox forced	Allow
all	Deny	Deny	Deny	Allow

Mode descriptors:

- **suggest** (read-only) — “the agent proposes; the user disposes”. Read tools (file reads, LSP queries, repomap dumps) execute; mutating tools (any `LevelEdit` / `LevelRun` / `LevelAll`) are REJECTED with `ErrApprovalRequired{Reason: "approval required: switch to auto-edit or higher"}`. Sandbox is irrelevant in this mode (no `LevelRun` ever fires). Network is ALWAYS deny for shell tools (academic — they’re denied at the gate before reaching the sandbox).
- **auto-edit** (edit, confirm-before-run) — file edits OK; shell commands prompt user via F19’s `PromptYesNo`. Sandbox is OPTIONAL in this mode (the user-facing prompt is the safety net). When the user approves a `LevelRun` call, the call goes through plain `os/exec` (no sandbox) — this is the “I’m at my desk, watching the agent” mode.
- **full-auto** (edit + run, sandbox-required, network DENY) — edits + shell commands OK without prompt; shell calls go through F14’s `sandbox.Manager.Execute(ctx, command, SandboxPolicy{NetworkAllowed: false})`. The sandbox MUST be available (Bubblewrap or native equivalent) — if F14’s `sandbox.Manager.Capabilities().HasSandbox == false`, the manager returns `ErrSandboxRequired` at startup time (NOT at `CheckApproval` time, so the user gets a clear error before any tool call). Network is FORCED DENY regardless of F14’s user-config network settings (anti-bluff: full-auto means “agent runs free, but sandboxed and offline”).
- **dangerously-bypass** (no checks) — every tool runs. Sandbox NOT forced. Network NOT denied. The user has accepted full responsibility. The slash-command output for `/approval set dangerously-bypass` includes a literal warning line (“This mode disables ALL approval checks. Press Ctrl-C now if you didn’t mean this.”) and a 2-second pause before the swap takes effect, to discourage accidental escalation. Unit test asserts the pause fires (via injected `time.Sleep` seam).

3.5 New external dependencies

Zero new dependencies. `gopkg.in/yaml.v3` is already direct (F20 promotion). `sync/atomic`, `errors`, `fmt`, `io/fs`, `os`, `path/filepath`, `strings`, `context` are all `stdlib`.

T08's verification step asserts `git diff go.mod` and `git diff go.sum` are both no-op after F21 implementation.

3.6 Per-tool RequiresApproval levels (T05 migration table)

The complete enumeration of every tool registered in `internal/tools/registry.go::buildToolList` (~30 tools) with its assigned level + rationale. T05's table-driven test pins this. Future tool additions that forget to override `RequiresApproval()` default to `LevelEdit` (safe-by-default) and the test fails until the table is updated.

Tool name	Level	Rationale
<code>read_file</code>	<code>LevelReadOnly</code>	Pure read; no state mutation.
<code>list_directory</code>	<code>LevelReadOnly</code>	Pure read.
<code>glob/grep/find</code>	<code>LevelReadOnly</code>	Pure read.
<code>repomap</code>	<code>LevelReadOnly</code>	Pure read of in-memory map.
<code>mapping_query</code>	<code>LevelReadOnly</code>	Pure read.
<code>lsp_query</code> (definition / references / hover / diagnostics)	<code>LevelReadOnly</code>	Pure read via LSP server.
<code>browser_read</code> (snapshot / network_requests)	<code>LevelReadOnly</code>	Pure read of in-memory browser state.
<code>write_file</code>	<code>LevelEdit</code>	Mutates filesystem.
<code>multi_edit</code>	<code>LevelEdit</code>	Mutates filesystem (batch).
<code>smart_edit</code> (F17)	<code>LevelEdit</code>	Mutates filesystem.
<code>notebook_edit</code>	<code>LevelEdit</code>	Mutates <code>.ipynb</code> file.
<code>mapping_edit</code>	<code>LevelEdit</code>	Mutates mapping store.
<code>plan_mode_edit</code> (F08)	<code>LevelEdit</code>	Mutates plan tree.
<code>git_*</code> (commit / checkout / branch)	<code>LevelEdit</code>	Mutates git state.
<code>shell</code> (raw os/exec)	<code>LevelRun</code>	Arbitrary process exec.
<code>shell_sandboxed</code> (F14)	<code>LevelRun</code>	Even sandboxed, still a process exec.
<code>interactive_command</code>	<code>LevelRun</code>	Interactive process.
<code>browser_navigate / click / type / evaluate</code>	<code>LevelRun</code>	Mutates browser state + can navigate to malicious sites.
<code>browser_run_code_unsafe</code>	<code>LevelRun</code>	Arbitrary JS exec.

Tool name	Level	Rationale
mcp_* (variable per server)	LevelEdit (default)	Conservative default; per-server overrides via MCP capability metadata in F21.5.
subagent_dispatch (F15)	LevelAll	Recursive agent spawn — can re-trigger any tool; LevelAll ensures only dangerously-bypass permits unattended use.
permission_rule_write (F02)	LevelAll	Mutates the permission engine itself — escalation vector.
ask_user (F19)	LevelReadOnly	Asks the human; not a state mutation.
theme_* (F20 internal helpers)	LevelReadOnly	Read-only previews.

Tools not in this table embed `approval.DefaultLevelEdit (LevelEdit)`. T05's `TestToolRegistryRequiresApprovalCoverage` enumerates every registered tool and asserts (a) it implements `RequiresApproval()`, (b) the returned level matches the table above, (c) no tool returns a level outside the 4-value enum.

3.7 YAML config schema

`~/.config/helixcode/approval.yaml` (Q2=C). File mode 0644 OK (the chosen mode is not sensitive — it's a posture string, not a credential). Schema:

```
# helixcode approval configuration
mode: auto-edit # one of: suggest | auto-edit | full-auto |
      dangerously-bypass
```

That's it. F21.5 may add per-tool overrides (`overrides: { write_file: read-only }`); v1 keeps the schema minimal.

Validation: - mode field missing → no error; selector continues to next precedence rung. - mode field set to unknown string → `ErrUnknownMode` wrapped with the offending value; selector logs WARN to stderr + falls through to default `suggest`. - File missing → no error; selector falls through. - File present but unreadable → log WARN + fall through. - File present but YAML parse error → log WARN + fall through.

3.8 Existing-code constraints

- **F02's** `internal/tools/permissions/` is **unchanged** structurally. F21 calls `policyEngine.Evaluate(req)` AFTER its own gate decided `Allow`; F02 retains final-deny authority. F02's existing test suite continues to pass without modification.
- **F14's** `internal/tools/sandbox/` is **unchanged** structurally. F21's `full-auto + LevelRun` path constructs a `sandbox.SandboxPolicy{NetworkAllowed: false}` and calls the existing `sandbox.Manager.Execute`. F14's existing test suite continues to pass.

- **F18's** `internal/render/` is **unchanged**. `/approval` slash output is rendered through F18's `RenderTextBlock` like every other slash command.
- **F19's** `internal/tools/askuser/` is **unchanged**. F21's `auto-edit + LevelRun` Prompt branch wraps the existing `Prompter.Prompt` (with a 2-choice `["yes", "no"]` Question) via a thin `YesNoPrompter` adapter — no new prompter implementation, no new package surface.
- **F20's** `internal/theme/` is **unchanged**. `/approval` status output uses the active `Styler` to colour the active mode (`RoleHighlight`) and the per-mode summary lines (`RoleInfo / RoleDim` for inactive modes).

4. Data flow

4.1 Startup

```
cmd/cli/main.go::run():
├─ flags := pflag.Parse() // --approval=<mode> wired into existing set
├─ approvalMode, source, err := approval.Select(approval.SelectorInput{
│   Flag:      flags.Approval,
│   Env:       os.Getenv,
│   ConfigPath: filepath.Join(xdgConfigHome, "helixcode/approval.yaml")
│   Filesystem: os.DirFS("/"),
│ })
│   └─ if err != nil:
│       log warning + use default ModeSuggest + SourceDefault
├─ // Build approval manager AFTER F02 (policy engine) + F14 (sandbox ma
├─ c.approvalMgr = approval.NewManager(approval.ManagerOptions{
│   Mode:      approvalMode,
│   Source:    source,
│   PolicyEngine: c.policyEngine, // F02
│   SandboxMgr:  c.sandboxMgr,    // F14
│   Prompter:   yesnoAdapter(c.prompter), // F19 wrapper
│   Logger:     c.logger,
│ })
├─ // Force-fail-fast: if mode == full-auto AND sandbox capabilities are
├─ if approvalMode == approval.ModeFullAuto && !c.sandboxMgr.Capabilitie
│   return fmt.Errorf("%w: full-auto requires bubblewrap or native san
├─ c.toolRegistry.SetApprovalManager(c.approvalMgr)
├─ // Register slash command
├─ c.commandRegistry.Register(commands.NewApprovalCommand(c.approvalMgr)
```

4.2 Per-call gate (the 4×4 matrix)

```
registry.Execute(ctx, name, params):
├─ tool := r.tools[name]
├─ // F21 gate (NEW; FIRST in chain)
├─ if r.approvalMgr != nil:
│   dec := r.approvalMgr.CheckApproval(ctx, tool, params)
│   switch dec.Action:
│       case ActionDeny:
│           return nil, fmt.Errorf("%w: %s", ErrApprovalRequired, dec.Reas
│       case ActionPrompt:
│           ok, err := r.approvalMgr.prompter.PromptYesNo(ctx,
```

```

        fmt.Sprintf("Approve %s with params %v?", tool.Name(), par
    if err != nil { return nil, err }
    if !ok {
        return nil, fmt.Errorf("%w: user declined at prompt", ErrA
    }
    case ActionAllow:
        // proceed
    if dec.SandboxRequired:
        params["_helix_sandbox_required"] = true
        params["_helix_sandbox_network_allowed"] = false
    // F02 hook (EXISTING; unchanged)
    if r.policyEngine != nil:
        // F02 evaluates and may return ActionDeny → final deny authority.
    // F14 dispatch (EXISTING; reads the marker if SandboxRequired was se
    if marker, ok := params["_helix_sandbox_required"].(bool); ok && mark
        // Re-route shell tool through sandbox.Manager.Execute(...).
    // Tool execute (EXISTING)
    return tool.Execute(ctx, params)

```

4.3 CheckApproval internals

```

func (m *ApprovalManager) CheckApproval(ctx context.Context, tool Tool, pa
mode := *m.mode.Load()
level := tool.RequiresApproval()
action := matrix[mode][level] // §3.4 4×4 table – pure lookup
switch action:
case ActionAllow:
    sandbox := mode == ModeFullAuto && level == LevelRun
    return Decision{Action: ActionAllow, SandboxRequired: sandbox}
case ActionDeny:
    return Decision{
        Action: ActionDeny,
        Reason: fmt.Sprintf("approval required: %s mode forbids %s-level
            mode, level, minimumModeForLevel(level)),
    }
case ActionPrompt:
    // The registry calls prompter – manager just signals.
    return Decision{
        Action: ActionPrompt,
        Reason: fmt.Sprintf("%s mode requires confirmation for %s-level
    }

```

4.4 F02 + F21 composition (final-deny rule)

F21's gate is the **first** check; F02's policy engine runs **after** F21 returned Allow. F02 may still return Deny (a permission rule forbids this specific path/command). F02's deny is final — F21's Allow does NOT override F02. The ordering matters because F02's rules are persistent (the user wrote them once); F21's mode is per-session. A user in `dangerously-bypass` who has a permission rule denying `rm -rf /etc` STILL gets the rule's deny.

Truth table (assume tool requires `LevelRun`):

F21 Decision	F02 Decision	Final	Why
Allow	Allow	execute	both layers permit
Allow	Deny	DENY	F02 rule overrides F21 mode
Allow	Ask	prompt via F02's askuser path	F02's confirmation flow runs
Prompt	(not reached unless user says yes)	depends on F02 after user yes	F21 Prompt → user yes → F02 evaluates
Deny	(not reached)	DENY	F21 short-circuits before F02

4.5 /approval slash command

```

/approval                → alias of /approval status
/approval status         → prints:
                           active = <mode>           [styled with Rol
                           source = <source>          [RoleInfo]
                           ----- per-level matrix for active mode --
                           read-only: allow
                           edit:      <action>        [RoleError if de
                           run:       <action>
                           all:       <action>
                           ----- next-step hint -----
                           to switch: /approval set <mode> [Role
/approval set <mode>     → atomic.Swap; new mode takes effect on next
                           If <mode> == "dangerously-bypass":
                           print warning line + 2-second pause befo
                           If <mode> unknown:
                           return ErrUnknownMode with the supported
/approval show <mode>    → prints ModeDescriptors[<mode>] (5-row matri
                           works for all 4 modes; <mode> == "all" → l

```

The slash command output is rendered through F18 + F20 (so per-line coloring per active theme), and obeys plain-mode-zero-color from F20.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Unknown mode in flag/env/YAML** — log WARN with the offending value + the supported list; fall through to next precedence rung. NEVER abort startup.
- **YAML parse error** — log WARN + fall through. NEVER abort startup.
- **full-auto selected but no sandbox** — abort startup with ErrSandboxRequired. The user gets a clear error before any tool call: “full-auto requires bubblewrap or native sandbox; switch mode or install bwrap”. Rationale: silently downgrading to auto-edit would be a security bluff (the user explicitly asked for sandbox; not getting it is a failure mode, not a fallback).
- **Tool with no RequiresApproval()** — does not compile (interface contract enforces it). The migration in T05 covers all existing tools.

- `/approval set with unknown mode` — return non-fatal error; slash command prints the supported list; mode unchanged.
- **Prompter cancelled (user pressed ctrl-c on y/n)** — `Decision.Deny` with reason “user cancelled at prompt”; `tool.Execute` NOT called.
- **F02 returns Ask while F21 already promoted to Allow** — F02’s ask flow runs (the user sees F02’s prompt for the *specific rule*, not F21’s mode-level prompt — they’re different questions).

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

Common bluff variants and their structural defences:

1. **(a) Approval mode loaded but never enforced** — the manager is constructed, `/approval status` shows the mode, but the registry never calls `CheckApproval`.
Defence: integration test wires a real `ApprovalManager` (`ModeSuggest, ...`) + a real `filesystem.WriteFileTool` + the real registry, calls `registry.Execute("write_file", {path: tmpFile, content: "x"})`, asserts (i) returned error wraps `ErrApprovalRequired`, (ii) `os.Stat(tmpFile)` returns `os.IsNotExist`. The Challenge’s PHASE-A mirrors this with byte-level path-existence assertion: the file MUST NOT exist after the call.
2. **(b) suggest mode says “would do X” but actually executes silently** — the gate produces `Decision.Deny` but the registry ignores it and proceeds. **Defence:** the integration test (a) above asserts BOTH the error wrap AND the absence-of-side-effect (`os.Stat == IsNotExist`). The test fails if the registry ignores the deny — even if the error happens to wrap correctly. The Challenge’s PHASE-A explicitly counts the `Execute` invocations on a fake tool (counter MUST be 0).
3. **(c) full-auto claims sandbox-on but RequiresApproval() doesn’t actually consult the sandbox** — the params marker `_helix_sandbox_required` is set but the registry’s F14 dispatch ignores the marker and runs through plain `os/exec`. **Defence:** integration test wires `ModeFullAuto` + a real `shell.SandboxedShellTool` + a real `sandbox.Manager` (with bubblewrap or skip-with-marker if not available) + invokes a command that would fail outside the sandbox (e.g., `cat /etc/shadow` succeeds without sandbox on most test setups but fails inside bwrap with `--ro-bind /etc /etc` excluded — choose a command whose success/failure is byte-distinguishable). Assert the captured `stderr` contains the sandbox-failure marker. The Challenge’s PHASE-C uses a similar invocation; the test is skipped (with SKIP-OK marker) only when bwrap is unavailable.
4. **(d) Mode change at runtime via /approval slash isn’t reflected in subsequent tool calls** — the slash mutates a struct field that `CheckApproval` doesn’t re-read (e.g., the mode is captured in a closure at `NewManager` time). **Defence:** unit test calls `manager.CheckApproval(ctx, fakeEditTool, ...)` with `ModeSuggest` → asserts `Decision.Deny`. Then calls `manager.Set(ModeAutoEdit)`. Then calls `manager.CheckApproval(ctx, fakeEditTool, ...)` again → asserts `Decision.Allow`. Same goroutine; no synchronisation games. The Challenge’s PHASE-D mirrors this in a real registry path.

Required real-execution criteria (define what “approval system works”):

1. **Suggest mode actually denies edits** — real `WriteFileTool` invoked through real registry returns `ErrApprovalRequired` AND target file does NOT exist on disk. Byte evidence: `os.Stat(target)` returns `os.IsNotExist == true`.
2. **Auto-edit mode allows edits, prompts on run** — real `WriteFileTool` succeeds (file exists with expected content); real shell tool with a yes-replying prompt runs; real shell tool with a no-replying prompt returns `ErrApprovalRequired`.

3. **Full-auto mode forces sandbox** — real shell tool + real sandbox manager → captured stderr contains sandbox-failure marker for a command that would succeed unsandboxed (or skip-with-marker if bwrap unavailable on test host).
4. **Runtime mode change** — real registry + `/approval set <mode>` invocation observable in the next `Execute` call's outcome.

Concrete forbidden-phrases anti-bluff smoke:

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/approval internal/commands/approval_command.go &&
echo BLUFF || echo clean
```

Must always print `clean`.

5.3 Concurrency

`ApprovalManager.mode` is an `atomic.Pointer[ApprovalMode]`. `CheckApproval` reads via `Load()`; `Set` writes via `Store()`. Concurrent reads + a single concurrent write are safe (atomics' guarantee). The 4×4 matrix is a constant `[numModes][numLevels]Action` array — no locks needed. The `PolicyEngine` + `SandboxManager` + `Prompter` are constructor-injected and read-only after construction; their own concurrency guarantees apply.

`/approval set <mode>` from the slash command runs in the main CLI goroutine; tool `Execute` may run concurrently in a worker goroutine. The atomic pointer ensures the worker either sees the old mode or the new mode, never a torn value.

5.4 CONST-042 (no-secret-leak) defensive log discipline

The approval YAML carries no secrets (the chosen mode is not sensitive). The manager's logger is restricted: at INFO level, log only `mode_changed: from=<X> to=<Y>` events; NEVER log the params map of a denied tool call (the params may contain a path or shell command body). A unit test scans `internal/approval/*.go` for `logger\.\bInfo\(. *\b(params|command|args|body)\b` matches and FAILs on any hit. This is consistent with F19's stance even though there are no actual secrets — log discipline is a programme-wide invariant.

CONST-042 specifically: when a mutating-command body would otherwise be logged at INFO time the approval prompt fires, the manager logs only the tool name + level, NOT the command body. The body MAY appear at DEBUG level (which is opt-in via env var); INFO-level logs MUST be content-free.

5.5 ModeDangerous 2-second pause

The 2-second pause before `/approval set dangerously-bypass swap` is a deliberate friction point. Implemented via an injectable `time.Sleep` seam (`ManagerOptions.SleepFunc func(time.Duration)`, default `time.Sleep`) so unit tests can assert (a) the seam is called with `2 * time.Second`, (b) the swap happens after the sleep returns. The seam is purely for test-time assertion; production behaviour is the literal 2-second pause + warning text. F21.5 may add a `--no-pause` flag for scripted CI use cases; v1 keeps the friction.

6. Testing

6.1 Unit (real value types; injected env closure; no mocks of stdlib)

Types (types_test.go): - TestApprovalMode_String_AllFour. -
TestApprovalMode_ParseMode_OK_KebabAndUnderscore. -
TestApprovalMode_ParseMode_Unknown_Err. -
TestApprovalLevel_String_AllFour. - TestAction_String_AllThree. -
TestModeDescriptors_TableShape_4x4Matrix — pin §3.4's table byte-for-byte. -
TestErrorSentinels_DistinctErrorsIs.

Selector (selector_test.go): - TestSelector_FlagWins_OverEnvAndConfig. -
TestSelector_EnvWins_WhenFlagEmpty. -
TestSelector_ConfigWins_WhenFlagAndEnvEmpty. -
TestSelector_DefaultSuggest_WhenAllEmpty. -
TestSelector_UnknownFlag_FallsThrough. -
TestSelector_UnknownEnv_FallsThrough. -
TestSelector_UnknownYAML_FallsThrough_LogsWarn. -
TestParseMode_AllFourValid (kebab-case + underscore variants). -
TestParseMode_Unknown_ErrUnknownMode.

Manager (manager_test.go): -
TestManager_CheckApproval_4x4Matrix_ExhaustiveTable — table-driven over
16 (Mode × Level) combinations; assert each Decision.Action matches §3.4. -
TestManager_CheckApproval_FullAutoRun_SandboxRequiredTrue. -
TestManager_CheckApproval_AutoEditEdit_AllowSandboxRequiredFalse. -
TestManager_CheckApproval_SuggestEdit_DenyWithReason. -
TestManager_CheckApproval_SuggestRun_DenyWithReason. -
TestManager_Set_AtomicSwap_NextCallSeesNewMode — call
CheckApproval(suggest, edit) → Deny; Set(autoEdit); call CheckApproval(autoEdit, edit) →
Allow. - TestManager_Set_DangerousBypass_PauseSeamFires — inject
SleepFunc; assert called with 2*time.Second. - TestManager_Mode_Source_Accessors. -
TestManager_NilPolicyEngine_OK — manager works without F02 wired (graceful for
tests). - TestManager_NilSandbox_FullAutoRun_ErrAtConstruct — but only in
main wiring; manager itself is nil-safe. -
TestDefaultLevelEdit_RequiresApproval_LevelEdit.

YAML loader (yaml_loader_test.go): -
TestLoadConfigFile_OK_AllFourModes. -
TestLoadConfigFile_Missing_NilNoErr. -
TestLoadConfigFile_Empty_NilNoErr. -
TestLoadConfigFile_ParseErr_ErrInvalidYAML. -
TestLoadConfigFile_UnknownMode_ErrUnknownMode.

Approval command (approval_command_test.go): -
TestApprovalCommand_Name_IsApproval. -
TestApprovalCommand_Status_PrintsActiveModeAndSource. -
TestApprovalCommand_Status_IncludesPerLevelMatrix. -
TestApprovalCommand_Set_OK_ChangesMode. -
TestApprovalCommand_Set_UnknownMode_Err. -
TestApprovalCommand_Set_Dangerous_PrintsWarning. -
TestApprovalCommand_Show_AllFourModes_RendersDescriptor. -
TestApprovalCommand_Show_UnknownMode_Err.

Tool RequiresApproval coverage (`registry_requires_approval_test.go`): -
`TestToolRegistryRequiresApprovalCoverage` — table-driven enumeration of every tool registered in `buildToolList`; assert each tool's `RequiresApproval()` matches §3.6's expected level.

6.2 Integration (`//go:build integration`)

`tests/integration/approval_test.go` (ALWAYS-runs; uses real F02 PE + real F14 SM + real F19 Prompter or fake-tty Prompter):

- `TestApproval_Integration_SuggestMode_DeniesWriteFile_FileNotCreated` — wire real registry + real `WriteFileTool` + `ModeSuggest`; call `registry.Execute("write_file", {path: tmpfile, content: "x"})`; assert error wraps `ErrApprovalRequired` AND `os.Stat(tmpfile)` returns `IsNotExist`.
- `TestApproval_Integration_AutoEditMode_AllowsWriteFile_FileCreated` — wire `ModeAutoEdit`; call same `Execute`; assert no error AND `os.ReadFile(tmpfile) == "x"`.
- `TestApproval_Integration_AutoEditMode_PromptsShell_UserYes_Runs` — wire `ModeAutoEdit` + a Prompter that returns yes; call shell tool; assert command output captured.
- `TestApproval_Integration_AutoEditMode_PromptsShell_UserNo_DeniesAndDoe` — Prompter returns no; assert `ErrApprovalRequired` AND command was NOT invoked (counter on a fake exec wrapper).
- `TestApproval_Integration_FullAutoMode_ShellGoesThroughSandbox` — wire `ModeFullAuto` + real `sandbox.Manager`; assert the params marker is set + `sandbox.Execute` is called (assertion via a Spy on the `SandboxManager`). Skip with SKIP-OK if bwrap unavailable.
- `TestApproval_Integration_FullAutoMode_NetworkDenied` — wire `ModeFullAuto` + a network-attempting shell command (e.g., `curl http://169.254.169.254/latest/meta-data/`) inside the sandbox; assert the curl returns network-failure exit code. SKIP-OK if bwrap unavailable.
- `TestApproval_Integration_DangerousMode_NoChecks` — wire `ModeDangerous`; assert all tools run without gate.
- `TestApproval_Integration_F02DenyOverridesF21Allow` — wire `ModeAutoEdit` (F21 says allow for edit) + F02 rule denying writes to `/etc/`; call `write_file` with `/etc/foo`; assert F02's Deny wins.
- `TestApproval_Integration_RuntimeModeChange_NextCallSeesNewMode` — wire `ModeSuggest`; first `Execute` returns Deny; call `manager.Set(ModeAutoEdit)`; second `Execute` returns Allow + file created.
- `TestApproval_Integration_FullAutoNoSandbox_FailsAtStartup` — construct manager with FullAuto but inject a `SandboxMgr` whose `Capabilities().HasSandbox=false`; assert `ErrSandboxRequired` returned at the construction-validation step.

6.3 Challenge (`challenges/p2-f21-codex-approval-modes/`)

Five-phase output skeleton (all always-run except sandbox phase which has skip-with-marker for bwrap-absent hosts):

```
=== APPROVAL-PHASE-A: SUGGEST-DENIES-EDIT (always runs) ===  
[PASS] constructed real ApprovalManager(ModeSuggest, ...) + real registry  
[PASS] called registry.Execute("write_file", {path: /tmp/p2f21-A, content:
```

```
[PASS] returned error wraps ErrApprovalRequired with reason "approval requ
[PASS] os.Stat("/tmp/p2f21-A") returns IsNotExist (no side-effect)
[PASS] WriteFileTool.Execute invocation count == 0 (gate short-circuited b
```

```
=== APPROVAL-PHASE-B: AUTO-EDIT-ALLOWS-EDIT-PROMPTS-RUN (always runs) ===
[PASS] constructed real ApprovalManager(ModeAutoEdit, ...) + yes-replying
[PASS] write_file succeeded; /tmp/p2f21-B content == "y"
[PASS] shell tool with prompter=yes ran; captured stdout contains expected
[PASS] re-wired ApprovalManager with no-replying Prompter
[PASS] shell tool returned ErrApprovalRequired with reason "user declined
[PASS] shell tool's exec wrapper invocation count == 0 in the no-replying
```

```
=== APPROVAL-PHASE-C: FULL-AUTO-FORCES-SANDBOX (skip-with-marker if no bwr
[PASS] sandbox.Manager.Capabilities().HasSandbox == true (bwrap detected)
[PASS] constructed real ApprovalManager(ModeFullAuto, ...) + real sandbox.
[PASS] called registry.Execute("shell", {command: "ip addr show"})
[PASS] params marker _helix_sandbox_required == true
[PASS] params marker _helix_sandbox_network_allowed == false
[PASS] sandbox.Manager.Execute spy invocation count == 1 (real sandbox pat
[PASS] command output present; stderr empty (or sandbox-allowed for this r
[PASS] separately: command "curl http://169.254.169.254" inside the same s
```

```
=== APPROVAL-PHASE-D: RUNTIME-MODE-CHANGE (always runs) ===
[PASS] manager mode == ModeSuggest at start
[PASS] write_file via registry → ErrApprovalRequired
[PASS] /tmp/p2f21-D does not exist
[PASS] called manager.Set(ModeAutoEdit) (simulates /approval set auto-edit
[PASS] write_file via registry → no error
[PASS] /tmp/p2f21-D content == "d"
[PASS] manager mode == ModeAutoEdit (after-state read)
```

```
=== APPROVAL-PHASE-E: F02-DENY-OVERRIDES-F21-ALLOW (always runs) ===
[PASS] constructed real ApprovalManager(ModeDangerous, ...) [F21 would all
[PASS] real F02 PE with rule denying write to /etc/
[PASS] called registry.Execute("write_file", {path: "/etc/p2f21-E", conten
[PASS] returned error wraps F02's denial reason (NOT F21's, because F21 sa
[PASS] /etc/p2f21-E does not exist (F02 final-deny authority)
```

```
SUMMARY: PHASE-A=5/5 PASS; PHASE-B=6/6 PASS; PHASE-C=8/8 PASS [or SKIP-OK:
          PHASE-D=7/7 PASS; PHASE-E=5/5 PASS
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable. Every phase records positive runtime evidence: invocation counters (PHASE-A counter == 0 proves gate fired), filesystem-state assertions (PHASE-A IsNotExist proves no side-effect), Prompter-replay equality (PHASE-B yes/no branches), sandbox-spy invocation count (PHASE-C count == 1 proves real sandbox path), runtime-state-difference (PHASE-D before/after mode change observable in tool outcome), F02-final-deny composition (PHASE-E).

7. Cross-platform

The 4×4 matrix is platform-independent. The Selector uses pure Go (os . Getenv via injected closure; os . DirFS for filesystem). The atomic.Pointer + sync primitives are stdlib.

The `full-auto + LevelRun` sandbox path delegates to F14, which is platform-conditional: bubblewrap on Linux, native backend on macOS (best-effort), native backend on Windows (best-effort). PHASE-C of the Challenge handles the platform variance via skip-with-marker semantics: if `sandbox.Manager.Capabilities().HasSandbox == false`, PHASE-C emits `SKIP-OK: #p2-f21-bwrap-absent` and PASS-counts the skip; otherwise it asserts the full sandbox path. The skip is honest (the SKIP-OK marker is a registered ticket; the test infrastructure does NOT silently green-pass).

`make prod cross-compile (linux/macos/windows)` is exercised in T08. No platform-specific code in F21's own files.

8. Out of scope (deferred)

- **Auto-promotion** (e.g., 3 successful auto-edits → propose full-auto) — F21.5; would need a per-session counter + a slash-command UX for the proposal.
- **Per-action audit log** beyond F02's existing audit — F21 reuses F02's audit for the (Mode + Level) decisions; a parallel approval-only audit stream is F21.5.
- **Multi-user RBAC** — out of scope (single-user CLI). F21.5 may add a `user` field to the YAML for shared-host scenarios.
- **Auto-write-back from /approval set** to the YAML config — F21.5; v1 deliberately keeps `/approval set` ephemeral so a misclick doesn't persist.
- **Per-tool YAML overrides** (e.g., `overrides: { write_file: read-only }`) — F21.5; v1 schema is `mode: <name> only`.
- **MCP per-server approval level** — v1 defaults all MCP tools to `LevelEdit`; F21.5 may read MCP capability metadata to derive per-server level.
- `--no-pause flag for dangerously-bypass` — F21.5; v1 keeps the 2-second friction unconditional (CI scripts that need it can use the `env var`).
- **Cobra helixcode approval subcommand** — F21.5 candidate (debug-only; the slash already covers the user case).
- **Per-call mode pinning** (e.g., “this single tool call runs in full-auto regardless of session mode”) — F21.5; v1 mode is session-scoped.
- `NO_APPROVAL=1 env-var bypass` — explicitly out of scope; would defeat the safety mandate. Users who want unattended runs use `dangerously-bypass` (which prints a loud warning).

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has FIVE always-run phases (PHASE-C uses skip-with-marker for bwrap-absent hosts; the marker is honest). Every phase records positive runtime evidence: invocation counters (PHASE-A: `counter == 0` proves gate fired before tool), filesystem-state assertions (PHASE-A: `os.IsNotExist`; PHASE-B/D: `os.ReadFile` content equality), Prompter-replay equality (PHASE-B: yes/no divergence), sandbox-spy invocation count (PHASE-C: `count == 1`), runtime mode-change observable in tool outcome (PHASE-D), F02 final-deny composition (PHASE-E). Every byte-evidence mismatch is a hard failure.
- **CONST-039** — Challenge at `challenges/p2-f21-codex-approval-modes/` + evidence harness at `tests/integration/cmd/p2f21_challenge/main.go`.
- **CONST-042 (No-Secret-Leak)** — approval YAML carries no secrets. The manager's logger NEVER logs the tool params at INFO level (the params may carry a path or shell command body); only mode-name + tool-name. A unit test scans `internal/approval/*.go` for `logger\.\bInfo\(. *\b(params|command|args|body)\b` matches and FAILs on any hit.

- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force; explicit user authorization is requested at T09 before pushing.
- **No-Mocks-In-Production (Universal Rule 2)** — ApprovalManager test seams are constructor-injected (PolicyEngine, SandboxMgr, Prompter, SleepFunc); no production code path uses a mock. Integration tests wire the **real** F02 PE, **real** F14 SM, real Prompter via F19; only the SleepFunc seam (for the dangerous-mode pause) and the test-only Spy on SandboxManager (a thin wrapper recording invocation counts) are test-injected.
- **CONST-033 (No host power management)** — F21 emits no shell commands of its own. The full-auto + LevelRun path routes USER-supplied commands through F14's sandbox, which already has a CONST-033 deny-list (suspend / hibernate / poweroff / reboot etc. — see F14 §5). F21 inherits F14's protection; F21 itself never invokes a power-state command.

10. Open questions resolved

Q	Answer	Resolution
Q1: 4 modes verbatim from codex	(A) suggest / auto-edit / full-auto / dangerously-bypass	Verbatim names from cli_agents/codex/; semantics per §3.4
Q2: selection precedence	(C) flag > env > config > default suggest	Mirrors F12's Selector pattern; pure function with injected env closure
Q3: per-tool gate	(B) Tool.RequiresApproval() ApprovalLevel	Tool knows what it does; default LevelEdit (safe-by-default); table-driven coverage test
Q4: F14 sandbox coupling	(A) suggest reject; auto-edit prompts run; full-auto forces sandbox+network deny; dangerously-bypass none	4×4 matrix in §3.4; SandboxRequired marker in Decision drives registry rewrite
Q5: surface	(A) /approval slash (status/set/show) + CLI flag + env var; NO cobra	Mirrors F20's slash+env pattern; runtime swap via atomic.Pointer

11. Non-obvious decisions (recorded for plan-time review)

1. **Separate** internal/approval/ **package** vs **extending** internal/tools/permissions/ — picked separate. F02 is rule-based (path globs); F21 is risk-posture-based (single enum gating whole categories). Conflating would either bloat F02 with mode predicates or bloat F21 with rule semantics. Keeping them separate lets each layer stay simple; final-deny composition (§4.4) gives the user F02's rule precision AND F21's mode breadth.
2. **F21 gate runs BEFORE F02** — chosen ordering. F21 is a coarse posture filter; F02 is fine-grained rules. Running F21 first short-circuits the “user is in suggest mode” case without asking F02 to enumerate every rule. F02 retains final-deny authority because its rules are persistent (the user wrote them once); F21's mode is per-session.
3. **Tool default LevelEdit (not LevelReadOnly or LevelAll)** — picked safe-by-default. A new tool that forgets to override gets the middle level (gated behind auto-

edit or higher). LevelReadOnly would be unsafe (a forgotten override could let mutations through suggest); LevelAll would be over-strict (every new tool would need dangerously-bypass). The migration test in T05 catches forgotten overrides for **existing** tools; the default catches **future** additions.

4. **Atomic-pointer mode swap** — chosen for runtime mode change. Lock-free, single writer (slash command goroutine) + many readers (tool worker goroutines). The 4×4 matrix is constant array data — no need to lock. Anti-bluff (d) (mode change not reflected) is structurally impossible because every CheckApproval call begins with `mode := *m.mode.Load()`.
5. **2-second pause before dangerously-bypass** — deliberate friction. Implemented via injectable SleepFunc seam. Unit test asserts the seam fires with `2 * time.Second`. F21.5 may add a `--no-pause` opt-out; v1 keeps the friction unconditional because the mode is genuinely dangerous and accidental escalation has happened in earlier revisions.
6. **full-auto + LevelRun forces network DENY** — chosen security posture. The mode is “agent runs free, but offline”. A user who wants network in full-auto can use `dangerously-bypass` (which is honest about its risk). Network-allowed full-auto would be a footgun: an agent that can both write files AND fetch arbitrary URLs is one prompt-injection away from exfiltration.
7. **F02 final-deny rule** — chosen composition. F21 is “the user opted into this risk posture for this session”; F02 is “this specific operation is forbidden by a rule the user wrote”. They compose monotonically: any deny denies. The unit test in T07 asserts both directions.
8. **Prompter is F19’s interface (wrapped via YesNoPrompter)** — reused, not re-implemented. Rationale: F19 already handles tty-detection, retry caps, and ctx cancellation correctly. Re-implementing would be duplicate code and a regression risk. The wrapper is a 5-line adapter that turns the y/n into F19’s 2-choice question.
9. **Mode descriptors are static data, not computed** —
ModeDescriptors[ApprovalMode] is a compile-time constant array. Rationale: the matrix is the source of truth; computing it at runtime would risk drift. The unit test pins each descriptor’s matrix row byte-for-byte against §3.4.
10. **Sandbox-required marker in params map** — chosen wire mechanism. F21 sets `params["_helix_sandbox_required"] = true`; F14’s existing dispatch reads the marker. Alternative (changing Tool.Execute’s signature to carry a SandboxPolicy) would require migrating every tool — a bigger change with no benefit (the marker is invisible to tools that don’t care about sandboxing).
11. **full-auto aborts startup if sandbox unavailable** — chosen failure mode. Silent downgrade to auto-edit would be a security bluff. The user explicitly asked for sandbox; not providing it is a failure, not a fallback.
12. **/approval set does NOT write back to YAML** — chosen ephemeral semantics. A misclick should not persist. The user who wants persistent mode change edits `~/.config/helixcode/approval.yaml` directly. F21.5 may add `/approval set --persist <mode>`.
13. **MCP tools default to LevelEdit** — conservative chosen default. F21.5 will read MCP capability metadata to derive per-server level. v1 over-gates rather than under-gates.
14. **Phase 2 first feature** — F21 is the inaugural Phase 2 feature. T01 advances PROGRESS.md from “Phase 1 complete” to “Phase 2 in flight”; the bootstrap evidence section in docs/improvements/06_phase_1_evidence.md is closed in F20-T09; a new docs/improvements/07_phase_2_evidence.md is created in F21-T01.